Software Architecture and Design Specification

Project: Hostel Management System

Version: 1.0

Team Number: 11

Authors:

Pradeep – PES2UG24CS357

Pavan Kumar H S – PES2UG24CS344

Prabhakar Kumar – PES2UG24CS355

Pavithra G C – PES2UG24CS346

Date: 04-10-2026

Status: Draft

Reference Documents: Software Requirements Specification (SRS) v1.0, dated 05-09-2026; Software Test Plan (STP) v1.0, dated 16-09-2026

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 11 (Pavithra G C, Pavan Kumar H S, Prabhakar Kumar, Pradeep)	Initial draft of the Software Architecture and Design Specification, derived from SRS v1.0 and STP v1.0.

Approvals

Role	Name
Architecture Lead (integration and review)	Pavithra G C
Architecture Contributor – Components and Pattern	Pavan Kumar H S
Architecture Contributor – Security and Data Stores	Prabhakar Kumar
Architecture Contributor – Design and Traceability	Pradeep
Product Owner / Faculty Reviewer	Sheela Mam

Table of Contents

Revision History
Approvals
1. Introduction
    1.1 Purpose
    1.2 Scope
    1.3 Audience
    1.4 Definitions
2. Document Overview
    2.1 How to use this document
    2.2 Related Documents
3. Architecture
    3.1 Goals & Constraints
    3.2 Stakeholders & Concerns
    3.3 Component (UML) Diagram
    3.4 Component Descriptions
    3.5 Chosen Architecture Pattern and Rationale
    3.6 Technology Stack & Data Stores
    3.7 Risks & Mitigations
    3.8 Traceability to Requirements
    3.9 Security Architecture
4. Design
    4.1 Design Overview
    4.2 UML Sequence Diagrams
    4.3 API Design
    4.4 Error Handling, Logging & Monitoring
    4.5 UX Design
    4.6 Open Issues & Next Steps
5. Appendices
    5.1 Glossary
    5.2 References
    5.3 Tools
Division of Work

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Hostel Management System (HMS) v1.0. It describes how the requirements stated in the Software Requirements Specification (SRS) are realised as a set of components, interfaces, data stores, and interactions, and it gives developers the design information needed to implement the system and testers the information needed to verify it against the Software Test Plan (STP).

This SAD is a direct continuation of the SRS and the STP. Every functional requirement (HMS-F-001 to HMS-F-017), non-functional requirement (HMS-NFR-001 to HMS-NFR-006), and security requirement (HMS-SEC-001 to HMS-SEC-006) is traced to the component and design element that realises it (Section 3.8), and every module described here corresponds to a test item listed in STP Section 2.

1.2 Scope

This document covers the user services of the Hostel Management System: administrator authentication and logout; student record management (add, view, update, remove, search); hostel and room management (add room, view room details, view available and occupied rooms, maintain room availability); room allocation (allocate, prevent allocation of a full room, view the allocated room, update an allocation, vacate a room); read-only student access to personal hostel and room information; and the security, error-handling, logging, and user-experience design that support these services.

The architecture addresses the web-based application built with Python (Django), MySQL, and HTML/CSS/JavaScript, as stated in SRS Section 2.4.

Excluded from this design, consistent with STP Section 4: hostel fee management, mess management, attendance management, complaint or maintenance requests, leave or gate-pass management, e-mail/SMS notifications, online payment gateways, mobile applications, multi-hostel deployment across distributed servers, load balancing, disaster recovery, and data migration from legacy records.

1.3 Audience

- Developers responsible for implementing the Hostel Management System.
- QA engineers and testers who verify the design against the STP and the acceptance tests in SRS Section 6.2.
- Security reviewers assessing the protection of student and hostel data.
- Instructors and faculty reviewing the mini-project.
- Maintenance teams and future developers extending the system.
- Hostel administrators and student representatives who wish to understand how the system works.

1.4 Definitions

Term	Definition
HMS	Hostel Management System
SRS	Software Requirements Specification
STP	Software Test Plan
SAD	Software Architecture and Design Specification (this document)
RTM	Requirements Traceability Matrix
FR / NFR	Functional Requirement / Non-Functional Requirement
Admin	Authorized person responsible for managing hostel records and room allocation
Student	A student whose hostel and room details are maintained in the system
Room Allocation	The process of assigning an available hostel room to a student
Room Availability	Information about whether a hostel room has available capacity
ADR	Architecture Decision Record – a short record of a decision, its rationale, and the alternatives rejected
UML	Unified Modeling Language
MVT	Model–View–Template – the structure used by the Django framework
ORM	Object-Relational Mapper – maps Python model classes to database tables (Django ORM)
RBAC	Role-Based Access Control
STRIDE	Threat categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
HTTP / HTTPS	Hypertext Transfer Protocol / its encrypted variant
TLS	Transport Layer Security
USN	University Seat Number – the unique identifier of a student
CI/CD	Continuous Integration / Continuous Delivery (GitHub Actions pipeline)

2. Document Overview

2.1 How to use this document

This document provides the architectural deliverables of the Hostel Management System: a UML component diagram (Section 3.3), component descriptions (Section 3.4), architecture decision records (Section 3.5), a STRIDE threat model (Section 3.9), UML sequence diagrams (Section 4.2), interface and API definitions (Section 4.3), and a requirement-to-component traceability matrix (Section 3.8).

Reader	Recommended sections
Developers	Sections 3.3 to 3.6 and 4.1 to 4.4 – structure, interfaces, data model, error handling
QA engineers / testers	Sections 3.8, 4.2, 4.3, and 4.4 – trace requirements to design elements and expected behaviour (complements STP Section 13.2)
Security reviewers	Sections 3.9 and 4.4 – threat model, access control matrix, logging rules
Instructors / faculty	The complete document; Section 3.8 demonstrates coverage of every SRS requirement
Maintenance teams	Sections 3.4, 3.5, 3.7, and 4.6 – module boundaries, decisions, risks, and open issues

Conventions: in diagrams and compact tables, requirement IDs are abbreviated (for example, F-010 means HMS-F-010 and SEC-003 means HMS-SEC-003). References written as "SRS §x" or "STP §y" refer to sections of those documents.

2.2 Related Documents

Document	Version / Date	Relationship to this SAD
Software Requirements Specification (SRS)	v1.0, 05-09-2026	Source of all functional, non-functional, and security requirements, user roles, constraints, and use-case diagrams (SRS §7).
Software Test Plan (STP)	v1.0, 16-09-2026	Defines test items (modules), test cases TC-001 to TC-017, TC-NFR-001 to 006, TC-SEC-001 to 006, and acceptance tests AT-001 to AT-016. Component names in this SAD match the test items in STP §2.
Requirements Traceability Matrix (RTM)	SRS §8 / STP §13.2	Extended in Section 3.8 with the component and design element that realise each requirement.
Use-Case Diagrams	SRS §7	Basis for the actors (Administrator, Student), the role-based access design, and the sequence diagrams.
Acceptance Tests	SRS §6.2	Behaviour that the design flows in Section 4.2 must satisfy.

3. Architecture

3.1 Goals & Constraints

Goals

- **Usability:** a simple, consistent interface so that common tasks can be completed with few steps (HMS-NFR-001).
- **Performance:** normal user requests are answered within 3 seconds (HMS-NFR-002).
- **Reliability and data integrity:** student, room, and allocation data remain accurate and consistent, and room availability always reflects the current allocations (HMS-NFR-003, HMS-F-016).
- **Availability:** the system is accessible during normal hostel administrative working hours (HMS-NFR-004).
- **Maintainability:** modules can be changed without affecting unrelated functions (HMS-NFR-005).
- **Scalability:** performance remains acceptable as the number of student and room records grows (HMS-NFR-006).
- **Security:** only authorized users reach administrative functions, stored data is protected, and input is validated (HMS-SEC-001 to HMS-SEC-006).
- **Testability:** every module can be tested at unit and integration level (STP §5.1).

Constraints

- Only authorized users may access administrative functions (SRS §2.5).
- Room allocation may be performed only when the room has available capacity (SRS §2.5).
- Student information must be stored accurately in the database (SRS §2.5).
- The system depends on the availability of the database server and the application server (SRS §2.5).
- The system is limited to hostel student and room management activities (SRS §2.5).
- Technologies follow SRS §2.4: Python with Django (selected from "Django or Flask"), MySQL, HTML/CSS/JavaScript, running on Windows or Linux and accessed with Google Chrome, Microsoft Edge, or Mozilla Firefox.
- No special hardware is required; the system is used from a desktop or laptop over a local network or the Internet using HTTP/HTTPS (SRS §3.2, §3.4).
- The project is built by a four-member team in sprints (STP §7), so proven built-in framework features are preferred to custom infrastructure.

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed in
Hostel Administrator	Fast and simple data entry; accurate room availability; protection against allocating a full room; secure login	§3.5, §3.9, §4.2, §4.5

Stakeholder	Concerns	Addressed in
Students	Privacy of personal data; correct view of their own hostel and room details	§3.9, §4.3.6
Developers	Clear modular structure, parallel work by four members, simple technology stack	§3.4, §3.5, §3.6, §4.1
QA engineers / testers	Traceability to test cases; testable modules; predictable error behaviour	§3.8, §4.3, §4.4
Security reviewers	Confidentiality, integrity, and authentication; coverage of threats	§3.9
Faculty / instructors	Conformance to the SRS and STP; documented design decisions	§2, §3.5, §3.8
Maintenance teams	Modifiability, documented interfaces, useful logs	§3.4, §4.3.7, §4.4

3.3 Component (UML) Diagram

Figure 3.1 shows the UML component diagram of the Hostel Management System. The system is a layered web application: the browser communicates over HTTP/HTTPS with Django templates and views, the views invoke the functional modules, and the modules access MySQL only through the Django ORM.

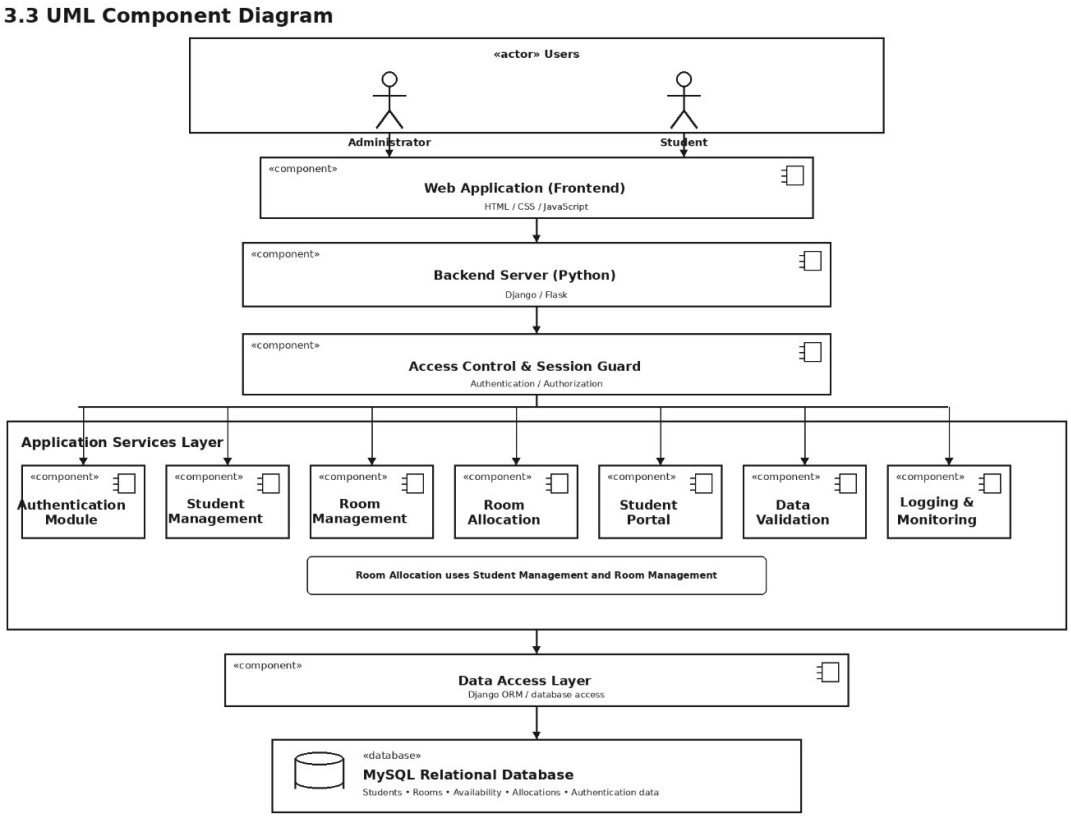


Figure 3.1: UML Component Diagram – Hostel Management System

Key relationships shown in the diagram:

- Every request passes through the Access Control & Session Guard before it reaches a module; only the login page and the health check are public.
- Room Allocation depends on Student Management (to confirm that the student exists) and on Room Management (to reserve or release capacity). Student Management and Room Management do not depend on Room Allocation, which keeps coupling low (HMS-NFR-005).
- Modules access data only through the Data Access Layer. A module reads another module's data through the ORM, but only the owning module writes it (single-writer rule).
- Data Validation and Logging & Monitoring are cross-cutting services used by all modules.
- The Student Portal is read-only and exposes only the logged-in student's own records (use cases in SRS §7.2).

3.4 Component Descriptions

The component names below correspond to the test items in STP Section 2.

Component	Implementation	Responsibilities	Requirements
Web Browser	Chrome, Edge, Firefox; HTML, CSS, JavaScript	Renders pages, collects user input, sends HTTP/HTTPS requests, holds the session cookie. Client-side checks are only a convenience; the server remains the authority.	NFR-001; SRS §3.1
UI Pages	Django templates; static CSS/JS	Login, dashboard, student pages, hostel and room pages, room availability, room allocation, room details, and search, with a consistent layout and clear messages.	NFR-001; SRS §3.1
Access Control & Session Guard	Django middleware and decorators (login_required, role checks), CSRF middleware	Admits only authenticated users to protected URLs; enforces Administrator and Student roles with default-deny; adds no-store cache headers; verifies CSRF tokens.	SEC-001, SEC-003, SEC-004, SEC-005
Authentication Module	accounts app; Django auth and sessions	Login, logout, credential verification, session creation and termination, generic failure message, password hashing.	F-001, F-002, SEC-001, SEC-002, SEC-005
Student Management Module	students app	Add, view, update, remove (after confirmation), and search student records; refuses removal while the student has an active allocation.	F-003, F-004, F-005, F-006, F-015
Room Management Module	rooms app	Hostel and room details; lists of available and occupied rooms; room details; owns the occupied_count of each room through reserve_bed and release_bed, so availability is always current.	F-007, F-008, F-009, F-016, F-017
Room Allocation Module	allocations app (services.py)	Allocate a room, prevent allocation of a full room, show a student's allocated room, change an allocation, and vacate a room, each in one atomic transaction; one active allocation per student.	F-010, F-011, F-012, F-013, F-014, F-016
Student Portal Module	portal app	Read-only access to the student's own details, hostel details, allocated room, and room information; restricted to the owner's record.	F-012; SRS §7.2

Component	Implementation	Responsibilities	Requirements
Data Validation	Django forms, validators, model constraints	Checks required fields, formats (10-digit phone, year 1 to 4, positive capacity), and uniqueness before anything is stored.	SEC-006, NFR-003
Logging & Monitoring	Python logging; middleware; /health/ view	Audit log of administrative actions, error log, response-time log, and a health check for availability monitoring.	NFR-002, NFR-004, SEC-004
Data Access Layer	Django ORM, models, migrations	Maps entities to tables, issues parameterised queries, runs atomic transactions, and enforces constraints and indexes.	NFR-003, NFR-006, SEC-004
MySQL Database	MySQL 8.x (hostel_db)	Persistent storage of users, students, hostels, rooms, allocations, and sessions.	NFR-003, NFR-004, NFR-006

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: layered (n-tier) architecture, deployed as one Django web application with a MySQL database. The four layers are the Presentation Layer, the Application/Business Layer, the Data Access Layer, and the Data Store. They map directly to Django's Model–View–Template structure: templates form the presentation layer, views, forms, and services form the application layer, and models with the ORM form the data access layer.

Layer	Responsibility	HMS elements
Presentation	Display pages, collect input, show messages	Web Browser, UI Pages (templates, CSS, JavaScript)
Application / Business	Request handling, access control, validation, business rules, transactions	Access Control, Authentication, Student Management, Room Management, Room Allocation, Student Portal, Data Validation, Logging
Data Access	Entity mapping, queries, constraints	Django ORM and models (User, Student, Hostel, Room, Allocation)
Data Store	Persistent storage	MySQL database hostel_db

Rationale:

- Separation of concerns: user-interface, business rules, and data access can each be changed without rewriting the others, which satisfies HMS-NFR-005.
- Testability: each layer can be unit-tested alone and the layers can be integration-tested together, matching the levels in STP §5.1.
- Fit to the team and schedule: four developers can work on different modules in parallel, and Django supplies authentication, validation, and ORM features out of the box.
- Performance and consistency: in-process calls and a single database keep responses fast (HMS-NFR-002) and allow allocation changes to be atomic (HMS-NFR-003, HMS-F-011).
- Right size: a single hostel with growing records is handled by indexes and pagination (HMS-NFR-006), not by distributing the system.

Alternatives rejected:

- Microservices were rejected as overly complex for a single-hostel system built by four students in three sprints. Network calls and distributed transactions would also endanger the consistency of room allocation, and the STP excludes load balancing and multi-hostel deployment.
- A separate single-page application with a REST API was rejected because SRS §2.4 specifies HTML/CSS/JavaScript with Django or Flask and the extra tier would double the work.
- Putting logic directly in views and templates without layers was rejected because it is hard to test and maintain.

Architecture Decision Records (ADRs)

ID	Decision	Rationale	Alternatives rejected	Status
ADR-001	Layered monolith using Django MVT	Simple, testable, fits team size and schedule	Microservices; unlayered code	Accepted
ADR-002	Django instead of Flask	Built-in authentication, sessions, ORM, form validation, and CSRF protection reduce custom and security-sensitive code; SRS §2.4 allows Django	Flask (needs extra libraries for the same features)	Accepted
ADR-003	MySQL as the relational database	Required by SRS §2.4; structured data with foreign keys, constraints, and transactions. SQLite is used only in development and CI	NoSQL store; SQLite for production	Accepted
ADR-004	Server-side sessions with role-based access control	Session can be destroyed on logout (SEC-005); no credentials held in browser scripts	Token-based (JWT) authentication	Accepted
ADR-005	Keep occupied_count on each room, changed by one atomic conditional UPDATE	Prevents over-allocation when two requests arrive together (F-011) and makes availability queries fast (NFR-002)	Counting allocations on every query (slower and race-prone)	Accepted
ADR-006	Server-rendered HTML pages	Meets SRS §3.1 with the fewest moving parts	SPA with REST API	Accepted
ADR-007	Cross-module changes only through service functions	Allocation changes room capacity only via reserve_bed and release_bed, keeping modules decoupled (NFR-005)	Writing other modules' tables directly	Accepted

3.6 Technology Stack & Data Stores

Area	Technology	Purpose
Programming language	Python 3.x	Backend development (SRS §3.3)
Web framework	Django (current LTS release)	Routing, views, forms, authentication, ORM, templates
Frontend	HTML, CSS, JavaScript; Django Template Language	User interface with automatic output escaping
Database	MySQL 8.x with the mysqlclient driver	Storage of student, hostel, room, allocation, and session data

Area	Technology	Purpose
Authentication and sessions	Django auth and session framework	Login, logout, PBKDF2 password hashing, server-side sessions
Transport security	HTTP/HTTPS (TLS)	HTTPS is used when the system is deployed over a network (SRS §3.4)
Logging	Python logging module	Audit, error, and performance logs
Testing	Django test runner / PyTest, Selenium, Postman, Apache JMeter	Unit, integration, system, security, and performance tests (STP §6.3)
CI/CD	GitHub Actions	Builds the project and runs the tests on every push
Version control and tracking	Git, GitHub, Jira	Source control, sprint board, defect tracking
Runtime environment	Windows or Linux; Django server for demonstration (a WSGI server such as Gunicorn if deployed); MySQL server	Hosting the application (SRS §2.4)

Data stores

Entity (table)	Key attributes	Constraints and notes
User (auth_user)	id, username, password hash, is_staff, is_active	username unique; is_staff marks an Administrator; passwords are never stored in plain text
Student (students_student)	id, usn, name, email, phone, department, year, user (optional link)	usn and email unique; phone must be 10 digits; year 1 to 4; indexes on usn and name; the link to User enables the Student Portal
Hostel (rooms_hostel)	id, name, block	name unique
Room (rooms_room)	id, hostel, room_number, capacity, occupied_count	unique (hostel, room_number); capacity at least 1; CHECK 0 ≤ occupied_count ≤ capacity. Available when occupied_count < capacity; listed as occupied when occupied_count > 0
Allocation (allocations_allocation)	id, student, room, status (ACTIVE / VACATED), allocated_on, vacated_on	At most one ACTIVE allocation per student; vacated records are kept as history; the room link is protected from deletion
Session (django_session)	session_key, data, expire_date	Server-side session; deleted on logout

Data integrity rules:

- occupied_count always equals the number of ACTIVE allocations of the room; the consistency test TC-NFR-003 checks this after every add, update, allocation, and vacation.
- Allocate, change, and vacate each run inside one atomic transaction, so a failure leaves no partial change.
- A student with an ACTIVE allocation cannot be removed; the administrator must vacate the room first.
- All input is validated before it is stored (HMS-SEC-006).

- The database is backed up regularly (mysqldump), supporting the recovery steps in STP §10.

Development and CI note: during Sprint 1 the CI pipeline runs the unit and integration tests on Django's default SQLite database for speed. MySQL is the target data store required by SRS §2.4 and is introduced through environment-based settings in Sprint 2. Because all data access goes through the ORM, no module code depends on the database engine; integration tests are re-run on MySQL before release (STP §6.2).

3.7 Risks & Mitigations

Risk	Mitigation	Related
Database or application server downtime makes the system unavailable	Regular database dumps; friendly error page; /health/ check during working hours; local backup MySQL instance for testing (STP §10)	NFR-004
Two requests allocate the last bed of a room at the same time	Atomic conditional UPDATE of occupied_count inside a transaction, plus a database CHECK constraint	F-010, F-011, NFR-003
Occupancy count and allocation records drift apart	Single-writer rule, service functions, transactions, and a consistency check in TC-NFR-003	F-016, NFR-003
Unauthorized access by typing a protected URL directly	Server-side login and role checks on every view with default-deny; student accounts receive 403 on administrative URLs	SEC-001, SEC-003
Pages shown from the browser cache after logout	no-store cache headers on protected pages; session deleted on logout	SEC-005
Slow responses as student records grow	Indexes on usn, name, and room number; pagination; select_related queries; JMeter tests	NFR-002, NFR-006
Tight coupling between modules makes changes risky	Layering rules, service interfaces, and integration tests across modules	NFR-005
Behavioural differences between SQLite (CI) and MySQL	Run integration tests on MySQL before release; avoid engine-specific code	NFR-003
Vulnerable or inconsistent dependencies	Pin versions in requirements.txt; the CI pipeline installs and tests from that file	SEC-004
Limited team experience with security design	Use Django's built-in security features; review against the STRIDE table in §3.9	SEC-001 to SEC-006

3.8 Traceability to Requirements

Each requirement of the SRS is mapped below to the component that realises it, the design element in this document, and the test case in the STP. Together with STP Section 13.2 this gives complete traceability from requirement to design to test.

Req ID	Requirement	Component(s)	Design element	Test case
HMS-F-001	Administrator login	Authentication	POST /login/ (§4.3.2); Figure 4.1	TC-001
HMS-F-002	Invalid login handling	Authentication	POST /login/ error path; §4.4	TC-002
HMS-F-003	Add student details	Student Management; Data Validation	POST /students/add/ (§4.3.3); Figure 4.2	TC-003

Req ID	Requirement	Component(s)	Design element	Test case
HMS-F-004	View student details	Student Management	GET /students/, GET /students/<id>/	TC-004
HMS-F-005	Update student details	Student Management	POST /students/<id>/edit/	TC-005
HMS-F-006	Remove student record	Student Management	POST /students/<id>/delete/ with confirmation; blocked while allocation is active	TC-006
HMS-F-007	Add room details	Room Management; Data Validation	POST /rooms/add/ (§4.3.4)	TC-007
HMS-F-008	View available rooms	Room Management	GET /rooms/available/; get_available_rooms()	TC-008
HMS-F-009	View occupied rooms	Room Management	GET /rooms/occupied/; get_occupied_rooms()	TC-009
HMS-F-010	Allocate room	Room Allocation; Room Management	POST /allocations/allocate/ (§4.3.5); Figure 4.3	TC-010
HMS-F-011	Prevent allocation of full room	Room Allocation; Room Management	reserve_bed() conditional UPDATE; CHECK constraint; Figure 4.3	TC-011
HMS-F-012	View allocated room	Room Allocation; Student Portal	GET /students/<id>/room/; GET /portal/room/	TC-012
HMS-F-013	Update room allocation	Room Allocation; Room Management	POST /allocations/<id>/change/	TC-013
HMS-F-014	Vacate room	Room Allocation; Room Management	POST /allocations/<id>/vacate/; Figure 4.4	TC-014
HMS-F-015	Search student records	Student Management	GET /students/?q=	TC-015
HMS-F-016	Update room availability	Room Management; Room Allocation	occupied_count maintained by reserve_bed() and release_bed()	TC-016
HMS-F-017	View room details	Room Management	GET /rooms/<id>/	TC-017
HMS-NFR-001	User-friendly interface	UI Pages	Screen design and accessibility rules (§4.5)	TC-NFR-001
HMS-NFR-002	Response time within 3 s	All layers; Database	Indexes, pagination, select_related; response-time monitoring (§4.4)	TC-NFR-002
HMS-NFR-003	Data accuracy and consistency	Data Access Layer; Room Allocation	Transactions, constraints, integrity rules (§3.6)	TC-NFR-003
HMS-NFR-004	System availability	Logging & Monitoring; MySQL	/health/ check, backups, error pages (§3.7, §4.4)	TC-NFR-004
HMS-NFR-005	Maintainability	Layered modules	Layering rules, single-writer rule, service interfaces (§3.5, §4.3.7)	TC-NFR-005
HMS-NFR-006	Scalability	Data Access Layer; MySQL	Indexes, pagination, query design (§3.6)	TC-NFR-006

Req ID	Requirement	Component(s)	Design element	Test case
HMS-SEC-001	User authentication	Authentication; Access Control	login_required on all protected views; Figure 4.1	TC-SEC-001
HMS-SEC-002	Invalid credentials rejection	Authentication	Generic failure message; throttling (§3.9)	TC-SEC-002
HMS-SEC-003	Admin access restriction	Access Control	RBAC matrix (§3.9.4); 403 for students	TC-SEC-003
HMS-SEC-004	Data protection	Data Access Layer; Access Control	POST-only changes, CSRF tokens, ORM queries, least-privilege database user (§3.9)	TC-SEC-004
HMS-SEC-005	Protected access after logout	Authentication; Access Control	Session deletion, no-store headers; Figure 4.5	TC-SEC-005
HMS-SEC-006	Input validation	Data Validation	Django forms, validators, model constraints (§3.4)	TC-SEC-006

3.9 Security Architecture

3.9.1 Security objectives and trust boundaries

The security objectives are those of SRS §5.2.1: confidentiality of student and hostel information, integrity of student, room, and allocation records, and authentication of administrative users. Two trust boundaries exist: between the user's browser and the application server (all input is untrusted and is validated on the server), and between the application server and the MySQL database (reached only through the ORM using a least-privilege database account).

3.9.2 Threat model (STRIDE)

Threat	HMS scenario	Mitigation in the design	Req
Spoofing	An attacker logs in as the administrator by guessing or stealing credentials	Credential verification against PBKDF2-hashed passwords; generic failure message; limit on repeated failed logins; session cookie marked HttpOnly (and Secure when HTTPS is used)	SEC-001, SEC-002
Tampering	Altering student, room, or allocation data through forged requests, crafted URLs, or injected SQL or script	CSRF tokens on every POST; changes accepted only through POST; server-side validation; parameterised ORM queries; template auto-escaping against XSS; HTTPS on networks	SEC-004, SEC-006
Repudiation	An administrator denies having deleted or changed a record	Audit log of every administrative action with user, action, record id, and time (§4.4)	SEC-004
Information disclosure	Student data shown to unauthorized users, leaked through error pages, logs, or cached pages	Access control on every view; Student Portal limited to the owner's record; DEBUG disabled and generic error pages; no sensitive data in logs; no-store cache headers; HTTPS on networks	SEC-003, SEC-005
Denial of service	Repeated login attempts or very large result lists slow the system	Limit on failed logins; pagination; database indexes; request time monitoring	NFR-002, NFR-004

Threat	HMS scenario	Mitigation in the design	Req
Elevation of privilege	A student account opens administrative URLs	RBAC enforced on the server for every view with default-deny; the administrator flag cannot be changed by users; students receive 403	SEC-003

3.9.3 Security requirement to control mapping

Req	Security requirement	Design control
HMS-SEC-001	Valid credentials before administrative access	login_required on every protected view; session created only after successful authentication
HMS-SEC-002	Reject invalid credentials	Same message "Invalid username or password." for any failure; no session created
HMS-SEC-003	Restrict operations to administrators	Role-check decorators; 403 for students; RBAC matrix below
HMS-SEC-004	Protect stored data from unauthorized change	CSRF tokens, POST-only changes, ORM queries, least-privilege database account, audit log
HMS-SEC-005	No protected access after logout	Session record deleted on logout; no-store headers prevent cached pages
HMS-SEC-006	Validate input before storing	Django forms, validators, and model constraints reject invalid or incomplete data

3.9.4 Role-based access matrix

Function	Not logged in	Student	Administrator
Login page, health check	Allowed	Allowed	Allowed
Logout	Not applicable	Allowed	Allowed
Dashboard	Redirect to login	Own portal only	Allowed
Add, update, remove students	Redirect to login	Denied (403)	Allowed
View and search all students	Redirect to login	Denied (403)	Allowed
Manage hostels and rooms; view available and occupied rooms	Redirect to login	Denied (403)	Allowed
Allocate, change, vacate rooms	Redirect to login	Denied (403)	Allowed
View own student and hostel details	Redirect to login	Allowed (own record)	Allowed (any record)
View own allocated room and room information	Redirect to login	Allowed (own record)	Allowed (any record)

4. Design

4.1 Design Overview

The Hostel Management System is designed as a layered, modular monolith. Each functional area is a separate Django app (module) with the same internal structure, which gives separation of concerns and easier maintenance:

```
hostel-management-system/
|-- hostel_project/     Django settings, root URLs, WSGI entry point
|-- accounts/           Authentication module (login, logout, dashboard)
|-- students/           Student Management module
|-- rooms/              Room Management module (hostels, rooms, availability)
|-- allocations/        Room Allocation module (services.py holds the rules)
|-- portal/             Student Portal (read-only views)
|-- templates/          Presentation layer: base layout and page templates
|-- static/             CSS, JavaScript, images
|-- tests/              Selenium, performance, and test-data scripts
|-- docs/               SRS, STP, SAD
|-- .github/workflows/  CI pipeline (ci.yml)
|-- requirements.txt, manage.py
```

Inside each module: models.py (data access), forms.py (validation), services.py (business rules), views.py (request handling), urls.py (routing), and tests.py (unit and integration tests).

Request processing flow:

1. The browser sends an HTTP/HTTPS request.
2. The URL router selects the view.
3. The Access Control & Session Guard checks authentication, role, and the CSRF token.
4. The view passes submitted data to a form, which validates it.
5. A service function applies the business rules inside a transaction where data changes.
6. The ORM reads or writes MySQL.
7. The view renders a template or redirects, and the outcome is logged.

Design rules:

- A layer calls only the layer directly below it; templates contain no business rules.
- Single-writer rule: only the owning module changes its tables. Allocation changes room occupancy only through reserve_bed and release_bed.
- Every view is protected by default; public views (login, health check) are listed explicitly.
- All changes to data are made with POST requests and the Post/Redirect/Get pattern.

Screen and URL map (SRS §3.1):

Screen	URL	Module	Access	Req
Login page	/login/	accounts	Public	F-001, F-002
Logout	/logout/	accounts	Logged in (POST)	SEC-005
Dashboard	/dashboard/	accounts	Administrator	SRS §7.3

Screen	URL	Module	Access	Req
Student list and search	/students/	students	Administrator	F-004, F-015
Add student	/students/add/	students	Administrator	F-003
Student details	/students/<id>/	students	Administrator	F-004
Update student	/students/<id>/edit/	students	Administrator	F-005
Remove student	/students/<id>/delete/	students	Administrator	F-006
Hostels and rooms	/rooms/, /hostels/add/, /rooms/add/	rooms	Administrator	F-007
Available rooms	/rooms/available/	rooms	Administrator	F-008, F-016
Occupied rooms	/rooms/occupied/	rooms	Administrator	F-009
Room details	/rooms/<id>/	rooms	Administrator	F-017
Room allocation	/allocations/allocate/	allocations	Administrator	F-010, F-011
Student's allocated room	/students/<id>/room/	allocations	Administrator	F-012
Change allocation	/allocations/<id>/change/	allocations	Administrator	F-013
Vacate room	/allocations/<id>/vacate/	allocations	Administrator	F-014
Student portal	/portal/, /portal/room/	portal	Student (own record)	SRS §7.2
Health check	/health/	core	Public	NFR-004

4.2 UML Sequence Diagrams

Five sequence diagrams show the main flows of the Hostel Management System. They cover authentication, student record creation with validation, room allocation including the full-room case, vacating a room, and logout with protected-page access.

4.2.1 Administrator login

Requirements: HMS-F-001, HMS-F-002, HMS-SEC-001, HMS-SEC-002. Test cases: TC-001, TC-002, TC-SEC-001, TC-SEC-002; acceptance tests AT-001, AT-002.

The browser posts the credentials with a CSRF token. The Login View checks that the fields are not blank and asks the Django authentication backend to verify the user against the stored password hash. On success a session is created and the administrator is redirected to the dashboard. On any failure the login page is shown again with one generic message and no session is created.

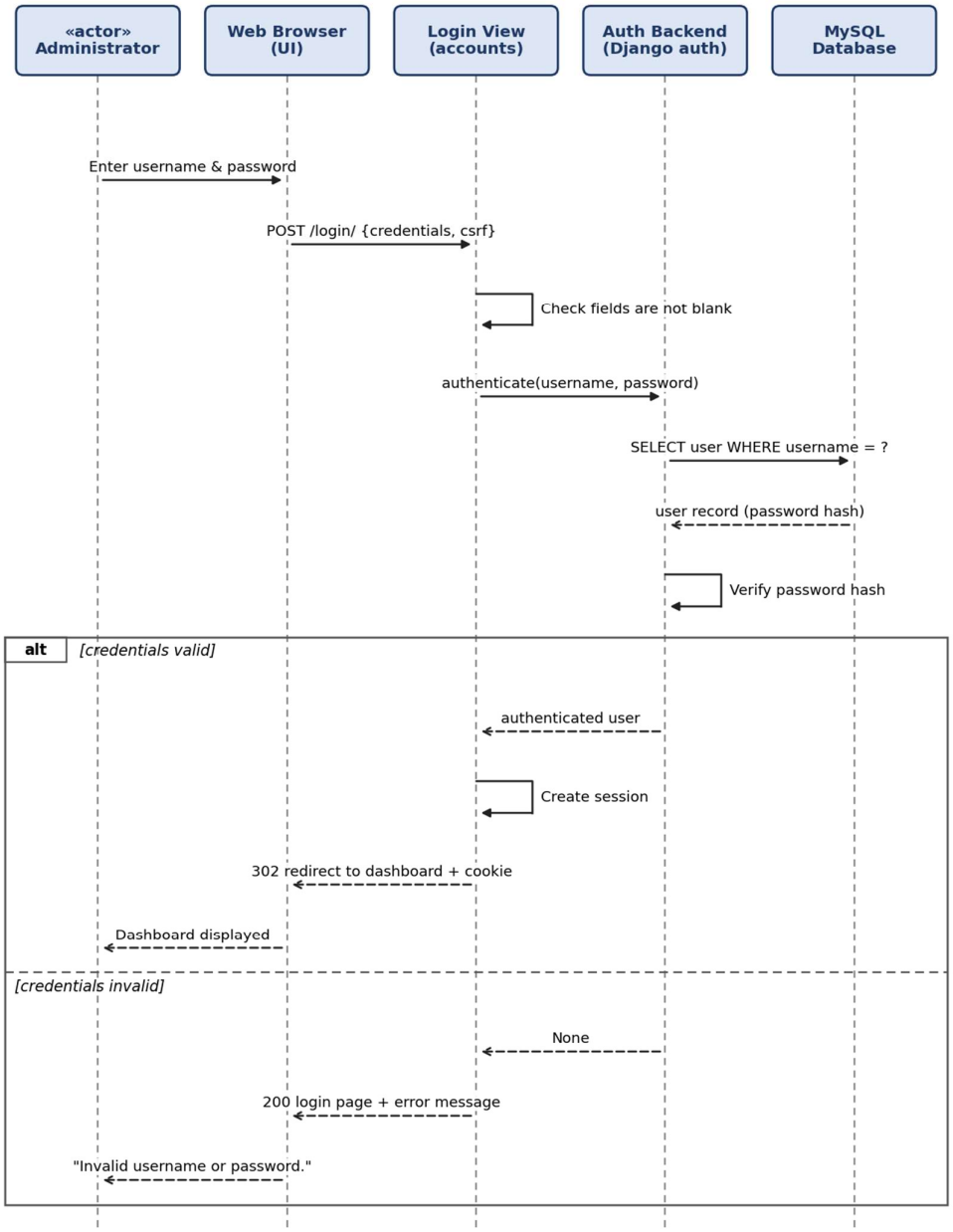


Figure 4.1: Sequence Diagram – Administrator Login

4.2.2 Add a student with validation

Requirements: HMS-F-003, HMS-SEC-003, HMS-SEC-006. Test cases: TC-003, TC-SEC-006; acceptance test AT-003.

After the session and administrator role are verified, the Student Form validates the data and checks that the USN and e-mail are unused. Only valid data is inserted into the database. Invalid or duplicate data causes the form to be shown again with field-level messages and nothing is stored.

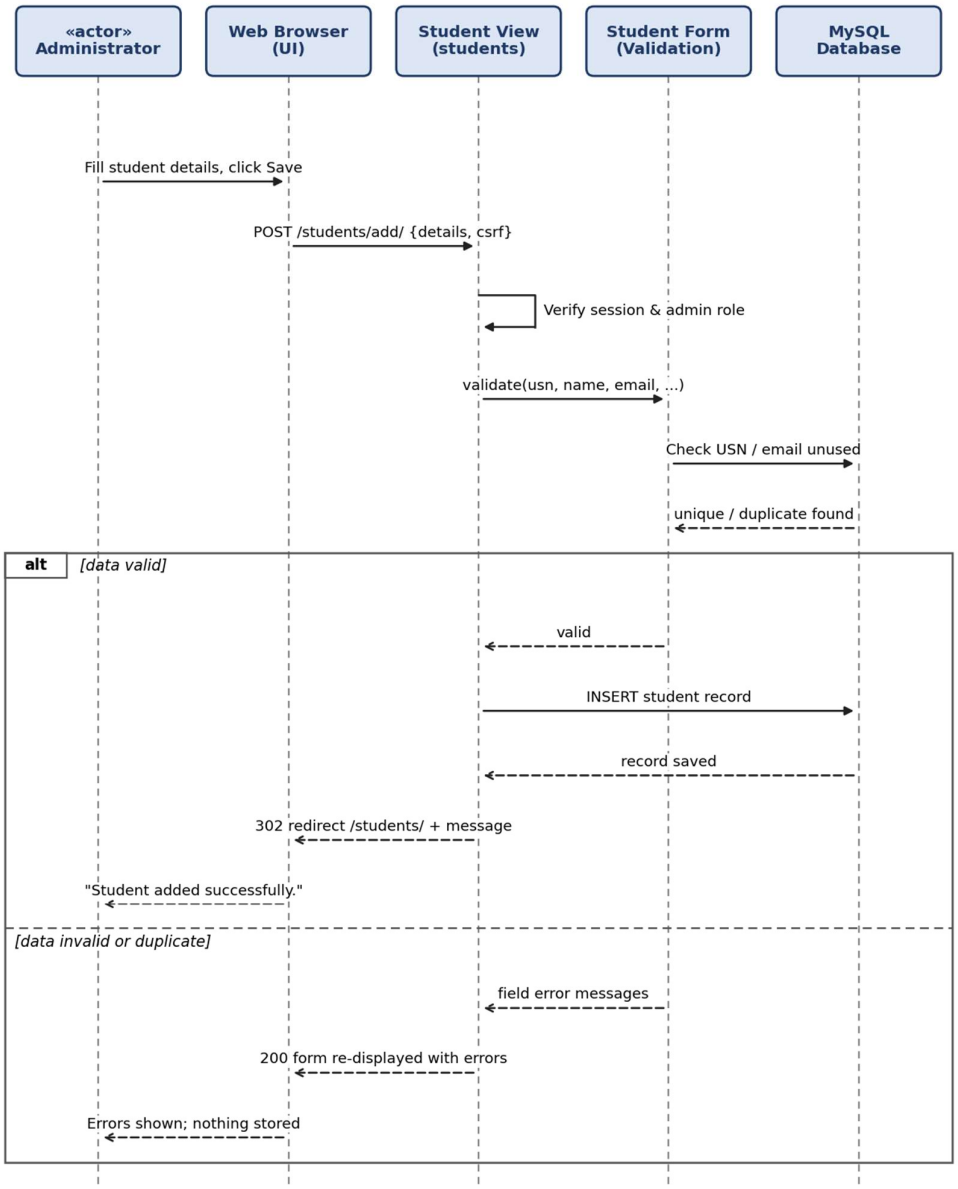


Figure 4.2: Sequence Diagram – Add Student with Validation

4.2.3 Allocate a room

Requirements: HMS-F-010, HMS-F-011, HMS-F-016, HMS-NFR-003. Test cases: TC-010, TC-011, TC-016, TC-NFR-003; acceptance tests AT-008, AT-009.

Inside one transaction the Allocation Module confirms that the student has no active allocation and asks Room Management to reserve a bed. Room Management performs a single conditional UPDATE that increases occupied_count only if it is still below capacity. If one row changed, the allocation record is inserted and the transaction is committed, so availability is updated in the same step (F-016). If no row changed, the room is full: the transaction is rolled back, nothing changes, and the administrator sees a clear message (F-011).

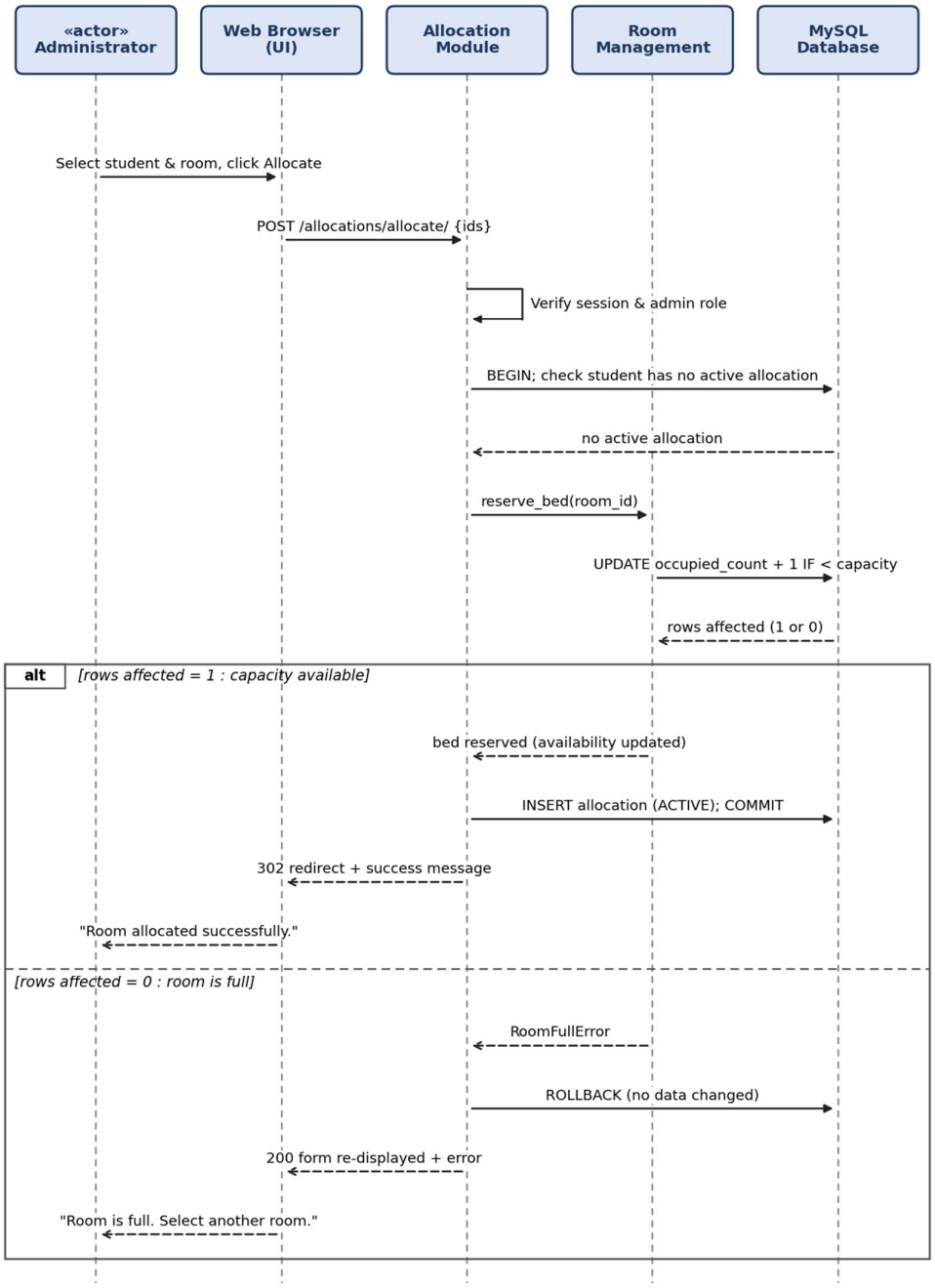


Figure 4.3: Sequence Diagram – Allocate Room (including full-room case)

4.2.4 Vacate a room

Requirements: HMS-F-014, HMS-F-016. Test cases: TC-014, TC-016; acceptance test AT-012.

The Allocation Module locates the active allocation, marks it VACATED with the date, and asks Room Management to release the bed. The occupied_count is decreased in the same transaction, so the room becomes available immediately. If no active allocation exists, the transaction is rolled back and an error message is shown.

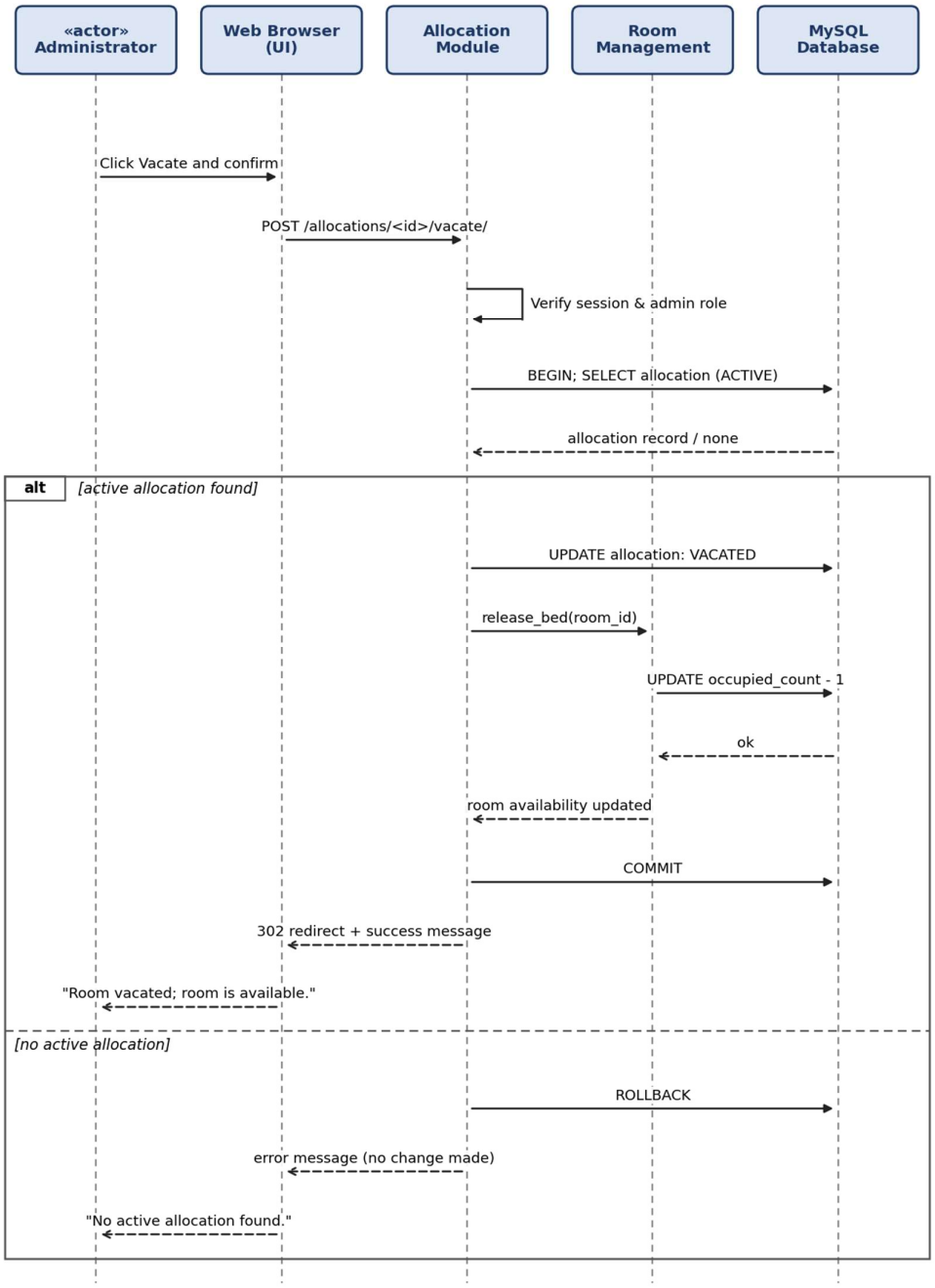


Figure 4.4: Sequence Diagram – Vacate Room

4.2.5 Logout and access to a protected page

Requirements: HMS-SEC-005, HMS-SEC-001, HMS-SEC-003. Test cases: TC-SEC-005, TC-SEC-003; acceptance tests AT-013, AT-016.

Logout deletes the session record and clears the cookie. Protected pages are sent with no-store cache headers, so pressing Back or typing a protected URL causes a new request. The Session Guard finds no valid session and redirects the browser to the login page, so authentication is required again.

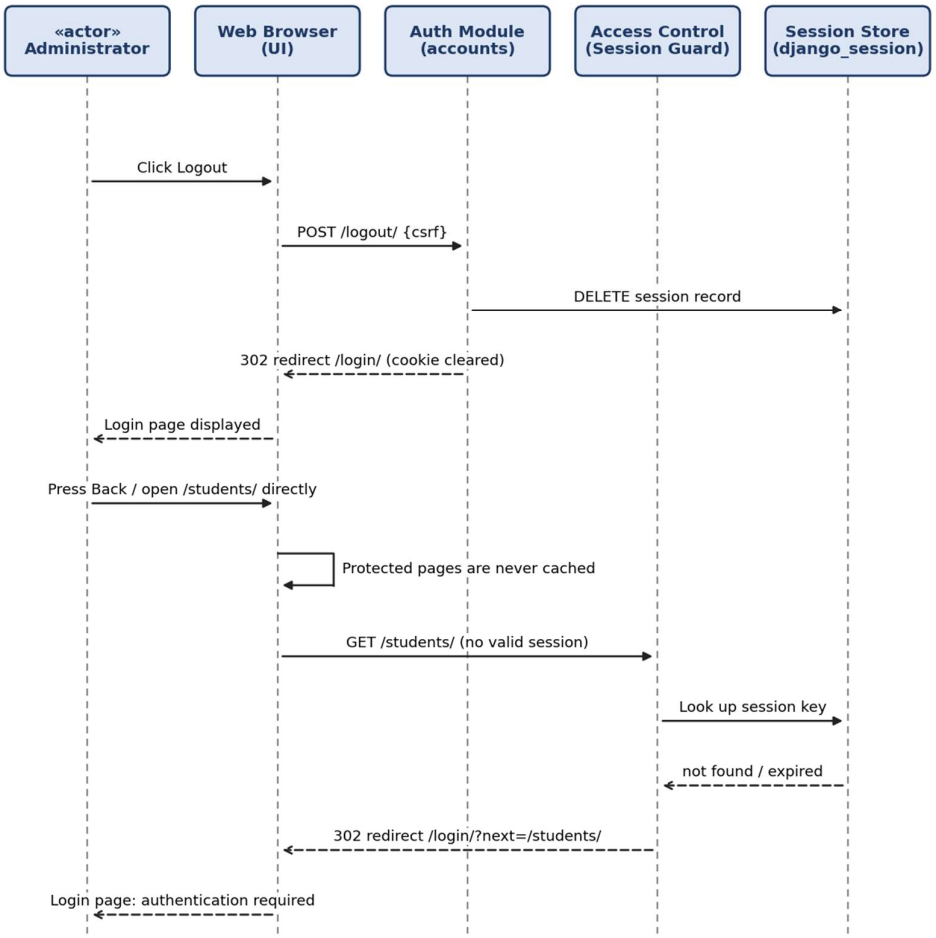


Figure 4.5: Sequence Diagram – Logout and Protected Page Access

4.3 API Design

4.3.1 Conventions

HMS is a server-rendered Django application, so its interfaces are HTTP endpoints that accept form data and return HTML pages or redirects. This section defines the interface of each component in the form endpoint, method, request, response, and errors.

- Requests that change data use POST with form fields and a CSRF token (csrfmiddlewaretoken). Reads use GET.
- Success: HTTP 302 redirect to the next page with a success message (Post/Redirect/Get).
- Validation or business-rule failure: HTTP 200, the form is shown again with error messages and nothing is stored.
- Not logged in: HTTP 302 to /login/?next=<url>. Logged in without permission: HTTP 403. Unknown record: HTTP 404. Unexpected failure: HTTP 500 or 503 with a generic page.
- Unless stated otherwise, endpoints require the Administrator role.

4.3.2 Authentication Module

Endpoint / Method	Request	Response	Errors

Endpoint / Method	Request	Response	Errors
GET, POST /login/ Public F-001, F-002, SEC-001, SEC-002	{username, password, csrf}	302 to /dashboard/ (Administrator) or /portal/ (Student); session cookie set	200 login page again: "Invalid username or password." "This field is required." (blank field)
POST /logout/ Logged in SEC-005	{csrf}	302 to /login/; session deleted and cookie cleared	403 if the CSRF token is missing or invalid 302 to /login/ if not logged in

4.3.3 Student Management Module

Endpoint / Method	Request	Response	Errors
GET /students/ F-004, F-015	Optional {q (name or USN), page}	200 paginated list of matching students	200 with "No students found." for an empty result
POST /students/add/ F-003, SEC-006	{usn, name, email, phone, department, year, csrf}	302 to /students/ with "Student added successfully."	200 with: "This field is required." "A student with this USN already exists." "Enter a 10-digit phone number." "Year must be between 1 and 4."
GET /students/<id>/ F-004	Path id	200 student details with current room	404 "The requested record could not be found."
POST /students/<id>/edit/ F-005	Same fields as add, {csrf}	302 to /students/<id>/ with "Student updated successfully."	200 with validation messages 404 if the student does not exist
GET, POST /students/<id>/delete/ F-006	{csrf}; GET shows the confirmation page	302 to /students/ with "Student removed."	200 "This student has an active room allocation. Vacate the room before removing the student." 404 if the student does not exist

4.3.4 Room Management Module

Endpoint / Method	Request	Response	Errors
POST /hostels/add/ F-007	{name, block, csrf}	302 to /rooms/ with "Hostel added successfully."	200 "A hostel with this name already exists."
POST /rooms/add/ F-007, SEC-006	{hostel, room_number, capacity, csrf}	302 to /rooms/ with "Room added successfully."	200 "Room capacity must be a positive whole number." 200 "This room already exists in the selected hostel."

Endpoint / Method	Request	Response	Errors
GET /rooms/available/ F-008, F-016	Optional {hostel, page}	200 rooms with occupied_count < capacity, showing capacity, occupied, and free beds	200 "No rooms with available capacity." when empty
GET /rooms/occupied/ F-009	Optional {hostel, page}	200 rooms with occupied_count > 0	200 "No occupied rooms." when empty
GET /rooms/<id>/ F-017	Path id	200 capacity, occupancy, status, and allocated students	404 "The requested record could not be found."

4.3.5 Room Allocation Module

Endpoint / Method	Request	Response	Errors
POST /allocations/allocate/ F-010, F-011	{student, room, csrf}	302 to /students/<id>/room/ with "Room allocated successfully."	200 "Room is full. Please select a room with available capacity." 200 "This student already has an active room allocation. Use Change Room instead." 404 if student or room is unknown
GET /students/<id>/room/ F-012	Path id	200 hostel, room number, and allocation date	200 "No room allocated yet." when none 404 if the student is unknown
POST /allocations/<id>/change/ F-013	{new_room, csrf}	302 with "Room allocation updated successfully."	200 "Room is full. Please select a room with available capacity." 200 "The student is already in this room." 404 if the allocation is unknown
POST /allocations/<id>/vacate/ F-014, F-016	{csrf}	302 to /rooms/<room id>/ with "Room vacated. The room is now available."	200 "No active room allocation found for this student." 404 if the allocation is unknown

4.3.6 Student Portal Module and System Interface

Endpoint / Method	Request	Response	Errors
GET /portal/ Student (own record) SRS §7.2	None (identity taken from the session)	200 own student details and hostel details	302 to login if not logged in 403 if the account has no linked student record
GET /portal/room/ Student (own record) F-012	None	200 own allocated room and room information	200 "No room allocated yet." when none 403 as above

Endpoint / Method	Request	Response	Errors
GET /health/ Public NFR-004	None	200 {status: "ok", database: "up"}	503 {status: "error", database: "down"}

4.3.7 Internal service interfaces

The modules cooperate through the following service functions, which keep the modules decoupled (ADR-007).

Service function	Behaviour	Returns / raises
RoomService. get_available_rooms(hostel=None)	Rooms whose occupied_count is below capacity (F-008)	Query set of rooms
RoomService. get_occupied_rooms(hostel=None)	Rooms whose occupied_count is above zero (F-009)	Query set of rooms
RoomService. reserve_bed(room_id)	Atomic UPDATE occupied_count + 1 WHERE id = room_id AND occupied_count < capacity (F-011, F-016)	Updated room; raises RoomFullError when no row changes
RoomService. release_bed(room_id)	Atomic decrease of occupied_count, never below zero (F-016)	Updated room
AllocationService. allocate_room(student_id, room_id)	In one transaction: check no active allocation, reserve_bed, create the ACTIVE allocation (F-010)	Allocation; raises StudentAlreadyAllocatedError, RoomFullError
AllocationService. change_room(allocation_id, new_room_id)	In one transaction: reserve_bed on the new room, release_bed on the old room, update the allocation (F-013)	Allocation; raises RoomFullError, NoActiveAllocationError, SameRoomError
AllocationService. vacate_room(allocation_id)	In one transaction: mark VACATED with date, release_bed (F-014)	Allocation; raises NoActiveAllocationError
AllocationService. get_active_allocation(student_id)	The student's ACTIVE allocation, if any (F-012)	Allocation or None
StudentService. search_students(query)	Case-insensitive match on name or USN (F-015)	Query set of students
authenticate(request, username, password)	Django authentication backend (F-001, F-002)	User or None

4.4 Error Handling, Logging & Monitoring

Standardized error messages

Condition	Message shown to the user	Behaviour
Invalid or unknown credentials	Invalid username or password.	Login page again (200); no session

Condition	Message shown to the user	Behaviour
Blank mandatory field	This field is required.	Form again (200); nothing stored
Invalid phone number	Enter a 10-digit phone number.	Form again (200); nothing stored
Year out of range	Year must be between 1 and 4.	Form again (200); nothing stored
Duplicate USN or e-mail	A student with this USN already exists.	Form again (200); nothing stored
Invalid room capacity	Room capacity must be a positive whole number.	Form again (200); nothing stored
Room is full	Room is full. Please select a room with available capacity.	Transaction rolled back; form again (200)
Student already allocated	This student already has an active room allocation. Use Change Room instead.	Form again (200)
No active allocation to vacate	No active room allocation found for this student.	Message shown; no change
Removing a student who has an active allocation	This student has an active room allocation. Vacate the room before removing the student.	Removal refused
Not logged in	Please log in to continue.	302 to /login/?next=<url>
No permission	You do not have permission to perform this action.	403 page
Record not found	The requested record could not be found.	404 page
Confirmation before removal	Are you sure you want to remove this student? This action cannot be undone.	Confirmation page before deletion
Unexpected or database error	Something went wrong. Please try again later.	500/503 generic page; details only in the server log

Logging

- Format: timestamp, level, module, user, action, record id, result.
- INFO: successful login and logout, and every create, update, delete, allocate, change, and vacate action (audit trail for non-repudiation).
- WARNING: failed logins, permission denials, and rejected allocations (room full).
- ERROR: unhandled exceptions and database failures, with stack traces kept only in the server log.
- Never logged: passwords, session keys, CSRF tokens, and full personal details such as phone numbers or e-mail addresses; the USN or record id is logged instead (STP §5.4).
- Log files are rotated, readable only by administrators of the server, and kept for a limited period.
- Debug mode is turned off outside development, so users never see stack traces.

Monitoring

Metric	Source	Target / use	Req
Response time of common operations	Request timing log; JMeter runs	Within 3 seconds	NFR-002
Application and database availability	/health/ check during working hours	Accessible in the defined operating period	NFR-004
Failed login attempts	Authentication log	Review repeated failures per user	SEC-002
HTTP 5xx error rate	Server log	Investigate every 5xx response	NFR-003
Rejected allocations (room full)	Application log	Informational; shows demand for rooms	F-011
Occupancy consistency (occupied_count equals active allocations)	Consistency check script and TC-NFR-003	Zero mismatches	NFR-003
Record volume and query time	Database slow-query log	No degradation as records grow	NFR-006

4.5 UX Design

The interface follows SRS §3.1: it provides clear labels, input fields, buttons, and messages so that users can perform every operation without difficulty (HMS-NFR-001). The design principles are simplicity (one task per page), consistency (the same layout, navigation bar, and button positions on every page), feedback (a success or error message after every action), and error prevention (confirmation before destructive actions and selection lists instead of free text where possible).

Screen	Main elements	Req
Login page	Username field, masked password field, Login button, error message area	F-001, F-002
Dashboard	Summary cards (students, rooms, available rooms, active allocations) and navigation links	SRS §7.3
Student details page (add / update)	Labelled form fields, Save and Cancel buttons, inline error messages	F-003, F-005, SEC-006
Student search and details page	Search box for name or USN, results table with View, Edit, and Remove actions, pagination, details view with current room	F-004, F-006, F-015
Hostel and room management page	Hostel form, room form (hostel, room number, capacity), list of rooms	F-007
Room availability page	Available and Occupied views, table of capacity, occupied, free beds, and a text status label	F-008, F-009, F-016
Room allocation page	Searchable student selector, available-room selector, Allocate button, Change and Vacate actions with confirmation	F-010 to F-014
Room details page	Capacity, occupancy, status, and list of allocated students	F-012, F-017
Student portal pages	Read-only profile, hostel details, allocated room and room information	SRS §7.2

Usability and accessibility rules:

- Body text of at least 16 px and a colour contrast ratio of at least 4.5:1.
- Every input has a visible label linked to it; error messages are written next to the field.
- Room status is shown with a text label (Available, Full), never by colour alone.
- All functions can be reached with the keyboard; the navigation bar is the same on every page.
- Layout suits desktop and laptop screens (SRS §2.4) and works in Chrome, Edge, and Firefox.
- Destructive actions (remove a student, vacate a room) always ask for confirmation.

4.6 Open Issues & Next Steps

ID	Open issue	Proposed resolution / next step
OI-1	CI runs on SQLite in Sprint 1 while SRS §2.4 requires MySQL	Add environment-based MySQL settings in Sprint 2 and re-run integration tests on MySQL before release
OI-2	How student portal accounts are created and who sets the first password	Administrator creates accounts with a forced password change; confirm with the faculty reviewer whether the portal is in the demonstration scope (SRS §2.3 says access depends on what is provided)
OI-3	Limit on repeated failed logins is designed but not yet built	Implement in Sprint 3 security hardening and test with TC-SEC-002
OI-4	HTTPS configuration for deployment over a network	Document the TLS setup and use HTTPS beyond localhost (SRS §3.4)
OI-5	Editing a room's capacity below its current occupancy	Reject the change: capacity must be at least occupied_count
OI-6	Final wireframes for the screens in §4.5	Team review before the Room Management and Allocation screens are built

Next steps by sprint:

- Sprint 1: Authentication, Access Control, Student Management, unit and integration tests, and the CI pipeline.
- Sprint 2: Room Management, Room Allocation, Student Portal, and the MySQL configuration.
- Sprint 3: security hardening, performance and scalability tests, regression testing, acceptance tests AT-001 to AT-016, and updating this SAD and the RTM (see the schedule in STP §7).

Future enhancements (outside the current SRS): report export to PDF or Excel, e-mail or SMS notifications, fee and mess modules, multi-hostel support, a room-request workflow for students, two-factor authentication, password reset by e-mail, bulk import of students from CSV, and a viewer for the audit log.

5. Appendices

5.1 Glossary

Terms HMS, SRS, STP, SAD, RTM, ADR, UML, MVT, ORM, RBAC, STRIDE, CSRF, XSS, HTTP/HTTPS, TLS, USN, and CI/CD are defined in Section 1.4. Additional terms used in the design:

Term	Meaning
Component	A modular part of the system with a defined responsibility and interface
Layered architecture	Structure in which each layer uses only the layer directly below it
Session	Server-side record that identifies a logged-in user between requests
Password hashing (PBKDF2)	One-way transformation used to store passwords without keeping the plain text
Transaction (atomic)	A group of database changes that either all succeed or are all undone
Foreign key	A column that refers to a row of another table
Index	A database structure that speeds up searches on a column
Pagination	Showing long lists in pages of limited size
Post/Redirect/Get	Pattern in which a successful POST is answered with a redirect, preventing duplicate submissions
Audit log	A record of who performed which action and when
WSGI	Standard interface between Python web applications and web servers

5.2 References

- Hostel Management System – Software Requirements Specification (SRS) v1.0, Team 11, 05-09-2026.
- Hostel Management System – Software Test Plan (STP) v1.0, Team 11, 16-09-2026.
- IEEE Std 42010 – Systems and software engineering: Architecture description.
- IEEE Std 829 – Software test documentation.
- OMG Unified Modeling Language (UML) specification.
- OWASP Top 10 and OWASP Application Security Verification Standard.
- NIST SP 800-160 – Systems Security Engineering.
- Microsoft STRIDE threat-modelling approach.
- Django documentation (authentication, ORM, forms, security) and MySQL Reference Manual.

5.3 Tools

- UML diagramming: PlantUML or draw.io (diagrams in this document can be edited or redrawn in either tool).
- Interface testing and documentation: Postman, Swagger.
- Development: Python, Django, MySQL, MySQL Workbench.
- Testing: Django test runner / PyTest, Selenium, Apache JMeter.

- Collaboration and automation: Git, GitHub, GitHub Actions, Jira.

Division of Work

The responsibilities for preparing this Software Architecture and Design Specification were divided equally among the four team members, following the same distribution used for the SRS and the STP.

Sl. No.	Team Member	Work Assigned
1	Pavithra G C	Introduction, Document Overview, Appendices, Revision History, and integration and review of the complete document
2	Pavan Kumar H S	Goals & Constraints, Stakeholders & Concerns, Component Diagram, Component Descriptions, and Architecture Pattern and Rationale
3	Prabhakar Kumar	Technology Stack & Data Stores, Risks & Mitigations, Security Architecture, and Error Handling, Logging & Monitoring
4	Pradeep	Traceability to Requirements, Design Overview, UML Sequence Diagrams, API Design, UX Design, and Open Issues & Next Steps

All four team members reviewed the complete Software Architecture and Design Specification and made the necessary corrections before submission. The responsibilities were distributed to ensure equal contribution towards the deliverable.